HydraKV: Adversarial AI Discovery of a Larger-than-Memory Key-Value Store that Outperforms FASTER on Skewed YCSB-A

Koushik Sen
EECS Department, UC Berkeley
ksen@berkeley.edu

Abstract

We report on HydraKV, a larger-than-memory key-value store that was designed, implemented, debugged, and hardened almost entirely by an AI agent framework (KISS Sorcar [15]) through a sequence of natural-language tasks, and on the *adversarial AI discovery* methodology used to keep the agent from overfitting the benchmark it was scored on. The setting is precisely specified: YCSB-A [2] (50% reads, 50% blind upserts, Zipfian $\theta = 0.95$) over 250M keys with 100-byte values ($\sim$25 GB of data) under a hard 8 GiB memory budget, on a 64-vCPU machine with eight local NVMe SSDs in RAID0. On this configuration Microsoft's FASTER [1], a state-of-the-art larger-than-memory store built for exactly this kind of point-operation workload, sustains 0.93 Mops/s with the same harness. The final HydraKV engine sustains 5.51 Mops/s on the same workload (median of three cold-cache runs: 5.51 / 5.51 / 5.53; 5.9$\times$), about 88% of the miss-bandwidth bound implied by the measured device ceiling and cache-hit rate. HydraKV is $\sim$2,050 lines of dependency-free C++17: an append-only O_DIRECT log, a fingerprint hash index, an admission-controlled write-back cache [8, 7], and a raw `io_uring` [13] read-miss path. None of these mechanisms is individually new; what we examine is whether an agent can find, combine, and correctly implement them under adversarial pressure.

The methodology alternates two agent activities: generating literature-informed synthetic workload variants intended to break the current engine (sparse SplitMix64 [14] key spaces, clustered and drifting hot sets motivated by published workload studies [4, 5]), and repairing the engine until it meets the goal on all variants, finishing with a held-out variant generated after engine work stopped and used only for measurement. The loop worked as intended at least once: the agent's first engine, which beat FASTER by $\approx$2$\times$ on the published trace, was OOM-killed by the first variant because it had baked in the trace's dense key space. The pre-hardening engine revision scored 3.87–5.67 Mops/s across all five workloads including the holdout; a subsequent hardening task (end-to-end workload tests across six build and sanitizer configurations, 92.84% line coverage, cross-model code review, and version bisection) fixed a delete/read resurrection race and a fingerprint-alias bug, and the hardened engine was then re-scored on the original workload at 5.51 Mops/s. We describe the engine, the evaluation and its scope limits, the measured throughput ceiling behind the agent's conclusion that a mid-task goal of 10 Mops/s was out of reach for this design on this machine, and what the exercise suggests about keeping optimizing agents honest.

Preprint.

1 Introduction

Point-operation key-value stores sit under much of today's internet infrastructure: session stores, feature stores, ML embedding services, social-graph caches [6, 4, 5]. The economically interesting regime is *larger-than-memory with a hard RAM budget*: the data set (here $\sim$25 GB) is a few times bigger than the memory the operator is willing to give the store (here 8 GiB), the SSD is fast but not free, and the access pattern is heavily skewed (Zipfian $\theta = 0.95$; a small hot set dominates). Update-heavy skew is the setting YCSB workload A models [2] and the setting FASTER [1] was engineered for: FASTER's hybrid log gives in-place updates for the in-memory hot tail and spills the cold majority to storage, and its paper reports throughput up to 160M ops/s when the working set fits in memory. When the data does *not* fit (250M keys, 100-byte values, an 8 GiB budget, reads that must return verbatim stored bytes), FASTER on our reference machine sustains 0.93 Mops/s. That number is the baseline this paper's artifact was asked to beat.

We did not set out to write a key-value store. We set out to test whether a general-purpose AI agent, given a machine, a benchmark harness, and a precise task specification, could produce one that is fast *and* robust, and whether it could be prevented from doing what optimizing agents notoriously do: overfitting to the one trace they are scored on. Systems that score candidate programs against a fixed evaluator, from evolutionary code search [16] to prompt optimization [17], face this evaluator-overfitting risk structurally; GEPA, for instance, guards against it with explicit train/validation/test splits. The vehicle here was KISS Sorcar [15], an open-source agent framework for long-horizon software engineering, driven by three successive natural-language tasks (Section 6): build an engine that beats FASTER by 2$\times$; then run an *adversarial discovery loop* that alternates between generating workload variants that break the engine and repairing the engine until it survives them, finishing with a held-out workload; then find and fix every bug, race, and redundancy the previous work left behind, without giving back the performance.

The surviving artifact, HydraKV, is $\sim$2,050 lines of self-contained C++17 (no dependencies beyond pthreads and raw Linux syscalls). Its architecture (Section 4) is a deliberately conservative composition of known ideas: an append-only 128-byte-slot log on O_DIRECT storage, in the family of log-structured designs [10, 1]; a cache-line-bucketed fingerprint hash index with lock-free CAS insertion, similar in role to FASTER's hash table but mapping keys to positions in an immutable log; a set-associative write-back RAM cache with SLRU-style segmented second-chance insertion [8], guarded by a probabilistic doorkeeper bitmap borrowed from TinyLFU [7]; and an asynchronous read-miss path built on hand-rolled `io_uring` rings [13] (the target machine has no liburing). The engine contains no constant derived from the workload's key count, key density, skew parameter, or trace files; its cache-entry geometry does assume the benchmark interface's small-value regime (Section 4).

On the original workload the final engine sustains 5.51 Mops/s (median of three cold-cache 30-second runs, 5.51 / 5.51 / 5.53; 5.9$\times$ FASTER's 0.93) while staying under the enforced 8 GiB resident cap, and passes an untimed read-back validation of all 250,000,000 keys. On the four adversarial variants (sparse keys, clustered hot sets, drifting hot sets, and a mixed held-out workload generated with fresh seeds after engine work stopped), the pre-hardening engine revision ("v4c") sustained 5.67, 5.56, 3.97, and 3.87 Mops/s (Section 5; the scored variant matrix was not re-run after hardening). A roofline-style argument (Section 5.3) shows the original-workload number sits close to the bound implied by the measured device ceiling (718k 4 KiB random-read IOPS) and the 77% cache-hit rate observed across every admission policy tried; the same arithmetic is why a mid-task goal of 10 Mops/s was, in our judgment, out of reach for this class of design on this machine. The agent reached that conclusion itself, documented it with measurements, and declined to circumvent it by exploiting the OS page cache outside its budget.

Contributions.

1. **An artifact.** A $\sim$2,050-line, dependency-free, larger-than-memory KV store that outperforms FASTER by 5.9$\times$ on one demanding, fully specified YCSB-A configuration, with the engine, variant generator, and test suite released alongside the paper.[1]
2. **A methodology.** *Adversarial AI discovery*: an agent-driven loop alternating workload-variant generation (kept within the operation mix, value size, key count, and skew regime of the target

[1] `kv_adversarial/` in `https://github.com/ksenxx/kiss_ai`: `hydra.cc` (engine), `gen_variant.cc` (trace generator), `test_hydra.cc` and `run_all_tests.sh` (test matrix).

workload, with shift axes motivated by published workload studies [4, 5]) with engine repair, terminated by a held-out variant. The loop demonstrably prevented one class of overfitting: it immediately killed a dense-key-index design that looked excellent on the published trace.
3. **A hardening protocol.** A separate task combining end-to-end workload tests across six build configurations (including ASan/UBSan, a TSan subset, and an `io_uring`-disabled fallback), cross-model read-only review, and version bisection. It removed real concurrency bugs, including a delete/read resurrection race that survived two plausible-looking fixes, while a performance gate (the original-workload median may not drop below its previous 5.48 Mops/s) kept the fixes from costing throughput.
4. **A measured ceiling.** A quantitative argument for why 10 Mops/s was unreachable for this design on this hardware, and a description of the reward-hacking opportunities (page-cache exploitation, value regeneration, trace precomputation) that the task specification and the agent's audits closed off.

Non-contributions and scope. HydraKV contains no individually novel data structure or replacement policy. It is a benchmark-shaped engine, not a deployable store: it lacks log compaction and crash recovery (Section 7), both core features of RocksDB [11] and FASTER. All absolute throughput numbers are specific to one machine, one harness, and one engine version each (Table 1); we compare against FASTER only on the original workload, the one configuration where both systems were measured. The claim we defend is narrower than "a better KV store": that a carefully instructed agent navigated this design space, implemented the result correctly enough to survive sanitizers and adversarial end-to-end tests, and was kept honest while doing so.

2 Problem Setting

The benchmark is fixed by a harness the engine may not modify. Each operation is a read or a blind upsert with equal probability (YCSB-A [2], unseeded RNG); keys are drawn Zipfian with $\theta = 0.95$ from 250,000,000 unique 64-bit keys; values are exactly 100 bytes, an 8-byte counter plus random bytes, stored verbatim. A load phase inserts all 250M keys (untimed); the scored phase is a fixed 30.0-second window driven by 16 worker threads pinned to one NUMA node, with reads issued asynchronously up to 256 in flight per worker. The score is the median of three runs, with `drop_caches` before each, so every scored run starts from a cold cache.

Correctness is enforced by three separate instruments, with different strengths. A separate untimed validation run reads back all 250,000,000 loaded keys and verifies full values against FNV-1a checksums. During scored runs, an auditor thread performs randomized cross-thread read-after-write checks, and the harness verifies the 8-byte counter of every read. The harness documentation is explicit that the random suffix bytes of *run-phase* overwrites are not re-verified during scoring (only their counters are, plus all load-phase values in validation), and equally explicit that exploiting this (e.g., regenerating or compressing values) is forbidden and treated as an integrity failure; the engine's own test suite uses deterministic full-value oracles precisely to cover this blind spot at smaller scales.

The machine is a GCP `n2-standard-64` (2× Cascade Lake Xeon, 64 vCPUs, 251 GB RAM) with eight 375 GB local NVMe SSDs in RAID0 (ext4, 512 KiB chunk). The engine's resident data is capped at 8 GiB (runs that exceed it score zero), and the whole process runs in a 14 GiB cgroup with swap disabled, the headroom covering the harness's own key and checksum arrays. Since ∼25 GB of live data cannot fit in 8 GiB, the engine must spill to SSD and cache a hot subset. The RAID0 device's random-read limits, measured with `fio` (4 KiB O_DIRECT random reads at high queue depth on the RAID device; the filesystem's 4096-byte logical block size makes 4 KiB the smallest read the engine issues), are 718k IOPS and 2.8 GB/s at ∼350 μs latency. FASTER, run on this machine with this harness at the same budget, sustains 0.93 Mops/s; that is the baseline used throughout. We report FASTER as configured by the benchmark's maintainers for this harness and did not tune it ourselves, a caveat that applies symmetrically: neither engine was tuned by the other's authors.

3 State of the Art and Its Limitations

LSM-trees. RocksDB [11] and its LSM [10] relatives are the workhorses of persistent KV deployment; the Facebook study [4] characterizes three of its production use cases in detail. LSMs earn their complexity on write-heavy traffic with range scans. For pure point operations on NVMe, however, the machinery becomes overhead: KVell [3] showed that on modern NVMe SSDs, where random and sequential bandwidth are comparable, LSM and B-tree stores become *CPU*-bound on compaction, sorting, and synchronization, and that an unsorted, shared-nothing design with batched device access recovers 2–$5\times$ throughput. Our workload has no scans, so sorted storage buys nothing.

Hash-index log stores. FASTER [1] is the natural baseline: a cache-optimized concurrent hash index over a hybrid log whose in-memory tail supports in-place updates, with epoch-based synchronization. Its headline numbers are for in-memory working sets; larger-than-memory reads go through asynchronous disk I/O and an admit-on-miss read-cache region. One structural limitation is documented in its own design: records fetched from disk are admitted unconditionally, so the one-touch tail of a Zipf distribution continuously evicts hot entries; admission control is absent. Beyond that we can only offer hypotheses for the measured gap, since we did not instrument FASTER's internals on this harness: candidate costs include index and log-page metadata competing with cached values for the 8 GiB budget at 100-byte records, and per-pending-operation completion overhead on the miss path. We flag these as unverified; the measured fact is the 0.93 Mops/s baseline itself.

Caching engines. The systems that take admission and scan resistance seriously are caches, not stores: segmented LRU [8], CLOCK-style second-chance recency [9], TinyLFU admission with its doorkeeper filter [7], and the engineering synthesis in CacheLib [6]. The Twitter trace study [5] found production skew often stronger and more dynamic than synthetic benchmarks assume, which is exactly what makes naive LRU fragile. But caches may drop data; a store may not. HydraKV's design question is therefore: how much of the cache literature's admission machinery can be bolted onto a log-structured store without breaking last-write-wins semantics under 16-way concurrency? Section 4 is the answer the agent converged on.

4 HydraKV Design

HydraKV is a single C++17 translation unit of $\sim$2,050 lines implementing the harness's `IKVStore` interface (`Read`, `ReadAsync`/`CompletePending`, `Upsert`, `RMW`, `Delete`, `Checkpoint`). At a high level:

Disk tier: an append-only slot log. Values live in fixed 128-byte slots in a single O_DIRECT file: 8 bytes of key, 4 of size, 4 of `prev` (a back-pointer forming per-key temporal chains that also route fingerprint aliases), and up to 112 bytes of data. Slots are staged into per-session 1 MiB chunk buffers and landed with sequential writes; landed pages are immutable except for tombstoning by `Delete`. There is no garbage collection: every write-back of an overwritten key appends a fresh slot, so under sustained update traffic the log grows without bound until an offline rewrite. Observed growth in the 30-second scored window is modest, but the design itself does not bound it, and the 32-bit position field caps the useful log at roughly 512 GiB. Sessions reserve disjoint 8,192-slot extents, which makes loading embarrassingly parallel but also means slot position is *not* a global write-order (a fact with correctness consequences discussed below).

Position index: fingerprint hashing with disk-verified reads. An open-addressing table of 8-byte words, each packing a 32-bit key fingerprint and a 32-bit slot position, in 64-byte buckets scanned with linear probing and populated by lock-free CAS. The table is the largest power of two fitting in a quarter of the memory budget (2 GiB at 8 GiB). Fingerprints alone collide constantly at this scale (millions of expected 32-bit pairs among 250M keys), so no read ever trusts the index: every candidate position is verified against the full key stored in the on-disk slot, and same-signature candidates are chained through `prev` pointers. The event that actually matters is rarer: two keys agreeing in both the 32-bit fingerprint and the table's 25 bucket-selection bits, an effective 57-bit signature that collides somewhere in the key population with probability $\approx$20% per run. An engine revision that treated such aliases carelessly could abort or bury a live key; the cross-model review of Section 6 caught this, and the shipped read path takes the highest-position verified match across

all candidate chains. One residual risk remains open: because sessions pre-reserve extents, position order is not globally temporal, and a crafted pair of same-signature keys whose *first* inserts race on different sessions can in principle create multiple index roots for which highest-position is not newest. The scored workload's single-threaded-per-key load cannot produce this, our alias tests (which craft same-signature keys deliberately) did not, and no fix was attempted; a store promoting this design to production would need one.

Memory tier: segmented write-back cache with a doorkeeper. The remaining budget (5.25 GiB at 8 GiB, after 2 GiB of index, 0.25 GiB of doorkeeper, and 0.5 GiB of slack) forms an 8-way set-associative cache of 120-byte entries, about 47M values. Entries inline values up to 101 bytes; larger values fall back to an unbounded in-memory overflow map, which is adequate for the benchmark's interface but forfeits the memory-budget property (Section 7). Two mechanisms keep the Zipf tail from churning the hot set. First, a *doorkeeper* in the spirit of TinyLFU [7], though simpler than TinyLFU proper: two rotating generations of budget-sized hash bitmaps (2×128 MiB), no frequency sketch. A read miss is admitted only if its key was already marked in a recent generation, so most one-hit wonders are read once and forgotten (hash collisions admit a small fraction early). Second, *segmented insertion* modeled on SLRU [8]: each set has four probation and four protected ways with one-bit second-chance recency (CLOCK [9]) rather than true LRU ordering; new admissions normally evict within probation only, and protected ways are populated by promoting entries re-referenced while recent, plus two pragmatic exceptions (empty protected ways may be filled directly, and a forward-progress fallback may evict a clean protected entry when a set is pathologically pinned). One-pass scans therefore churn roughly the probation half of the cache rather than all of it. Blind upserts of uncached keys are absorbed as dirty entries and written back lazily; upserts of cached keys update in place under a per-set spinlock with a version bump.

Write-back: cleaners and pin-ownership landing. Four background cleaner threads scan probation ways for *cold* dirty entries (hot dirty keys stay cached and coalesce their overwrites), stage them into flush chunks, and land them with sequential appends, after which the index is repointed old→new position by CAS. Landing is guarded by a *pin-ownership* protocol: a staged entry is pinned in a `FLUSHING` state, its 32-bit version field overwritten with a fresh staging token (unique until 32-bit wrap, an accepted and documented risk horizon); the flush record may touch the index only if the exact pin (key, state, token, old position) is still present under the set lock at landing time. A concurrent upsert or delete invalidates the pin, and the orphaned record lands as dead bytes in the log but never reaches the index. This protocol is what finally closed the delete/read resurrection race of Section 6; two earlier epoch-counter designs looked correct and were not, because a relocation staged *before* a delete could land *after* the tombstone pass.

Read misses: raw `io_uring`. `ReadAsync` completes cache hits inline; misses are sharded to eight dedicated I/O threads, each owning a hand-rolled `io_uring` ring (queue depth 128; direct `io_uring_setup`/`io_uring_enter` syscalls with explicit acquire/release barriers on the shared rings [13], since the machine lacks liburing) driving a state machine that reads the 4 KiB page, verifies the key, walks alias chains if needed, consults the doorkeeper, and completes the harness's read slot. If ring setup fails the engine degrades to a 48-thread synchronous `pread` pool; a runtime environment knob (`HYDRA_NO_URING`) forces this path so tests cover it. `Delete` is supported via a deleted-key registry whose fast path is a single atomic load when no delete is in flight; a deletion mark persists until the key is re-upserted. `Checkpoint` flushes the calling session's partial chunk and all currently-dirty cache entries with 32 parallel workers, then `fdatasync`s; it cannot drain *other* live sessions' partially filled chunks, so its durability contract assumes quiescent or stopped sessions; this is a stated precondition, not a discovered bug.

Sizing and workload assumptions. Index, doorkeeper, and cache are all sized from `mem_budget_bytes`; keys are hashed with a SplitMix64-style finalizer [14] so dense and sparse key spaces behave identically; the engine never reads the trace files. Two classes of fixed constants remain: the slot/entry geometry (128-byte slots, 120-byte entries, 101-byte inline cap) is chosen for the interface's small-value regime and would need re-derivation for larger values, and the thread-pool widths (8 rings, 4 cleaners, 48 fallback readers, 32 checkpoint workers) are hand-tuned to this machine class.

Table 1: Throughput on the reference machine (median of three 30 s cold-cache runs, 16 workers; per-run values where recorded). The adversarial matrix (lower block) was scored with the pre-hardening v4c engine and frozen when the hardening task began; only the original workload was re-scored with the final engine. FASTER was measured only on the original workload, so no speedup is reported for the variants. The holdout used fresh seeds, was generated after all engine work stopped, and received no tuning; its first measurement launch was killed externally mid-run after scoring 3.86 and was relaunched without any engine change (runs 3.87 / 3.87 / 3.21). StoreRSS stayed ≤ 8 GiB on every scored run.

System	Workload	Mops/s	vs. FASTER
FASTER	original (Zipf θ=0.95)	0.93	1.0×
HydraKV, first cut	original	1.85	≈2×
HydraKV, final	**original** (runs 5.51 / 5.51 / 5.53)	**5.51**	**5.9**×
HydraKV, v4c	original	5.48	—
HydraKV, v4c	V1: sparse SplitMix64 keys	5.67	—
HydraKV, v4c	V2: clustered hot set	5.56	—
HydraKV, v4c	V3: drifting hot set	3.97	—
HydraKV, v4c	Holdout: 60/25/15 mix, θ=0.92	3.87	—

5 Experimental Evaluation

5.1 Main results

All runs use the unmodified harness, cold caches, the enforced 8 GiB resident cap, and the scored environment (100-byte values, asynchronous reads with pipeline depth 256, integrity auditor on). Every configuration first passes the untimed validation run: all 250,000,000 keys read back with correct checksums, zero missing, zero wrong, zero cross-thread audit failures.

The final engine's three runs on the original workload were 5.51 / 5.51 / 5.53 Mops/s. For calibration: the first engine the agent wrote (a dense flat-array index, Section 6) reached 1.85 Mops/s, just under 2× FASTER; the remaining 3× accumulated across the adversarial loop's redesigns: log-append write-back replacing in-place 4 KiB read-modify-writes, doorkeeper admission, segmented insertion, and the `io_uring` miss path. Each was adopted or reverted after measurement on the scored configuration, but the measurements were taken on an evolving code base, so the per-mechanism scores (Section 5.5) are an engineering log, not a controlled ablation.

5.2 Adversarial variants and the holdout

The variant generator (released with the engine) produces load/run trace pairs in the harness's format. All generated variants draw keys as SplitMix64 hashes of insertion indices (sparse, uniformly scattered 64-bit keys, unlike the published trace's dense key space) and vary three knobs: skew θ, hot-set geometry, and RNG seeds. The geometries are *scrambled* (hot keys scattered arbitrarily), *clustered* (hot keys drawn from contiguous *insertion-index* ranges, which after hashing concentrates them in on-disk placement rather than in external key ranges; a deliberate placement-locality stressor, *inspired by* but not an implementation of the key-range locality documented at Facebook [4]), *drift* (the hot set crossfades between epoch-specific scrambles across the run, an author-designed temporal-shift stressor motivated by the observation that production popularity is dynamic [5]), and *mix* (a seeded blend of the three). Variants were constrained to stay within the target workload's operation mix, value size, key count, and skew regime, so that a variant could not "win" by being physically incapable of high throughput (a uniform-random variant would prove nothing on this device). These are literature-informed synthetic stressors, not trace replays, and their fidelity to production workloads remains a threat to validity.

V1 (sparse keys) immediately falsified the first engine: its flat 4-byte-per-key position array assumed the dense $[0, N)$ key space of the published trace, and on sparse 64-bit keys the process was OOM-killed. The fingerprint hash index that replaced the array costs memory and CPU but generalizes to arbitrary keys. V2 and V3 were then generated against the improved engine. The clustered variant scored slightly higher than the scrambled original (5.56 vs. 5.48); the difference is about 1.5% and we did not collect the I/O-level measurements needed to attribute it. Drift (V3) is genuinely harder

(3.97): a moving hot set halves the effective lifetime of every admission decision, the measured hit rate falls accordingly, and no engine change we tried recovered the difference without hurting the stationary workloads. The holdout (fresh seeds, mixed geometry, $\theta = 0.92$, generated only after all engine work stopped) landed at 3.87 Mops/s, slightly below the drift variant, consistent with its drift component dominating its difficulty. One measurement caveat, disclosed in Table 1: the holdout's first `run.sh` launch was killed by an external signal mid-way (after a 3.86 first run) and the measurement was relaunched; no engine or configuration change occurred between launches, so the no-adaptation property holds, though the measurement did span two launches.

5.3 The throughput ceiling, and the 10 Mops/s goal

Midway through the discovery loop the target was raised from $2\times$ FASTER to a flat 10 Mops/s. The agent's eventual response was a ceiling argument; we reproduce it here with its scope made explicit. Each scored run starts cold and lasts 30 s. At 50% reads, T Mops/s implies $T/2$ million read ops/s, of which a fraction $1-h$ miss the cache and, in this design, cost one 4 KiB device read each. The device delivers at most 718k such reads/s. Sustaining T=10 therefore needs $h \geq 1 - 0.718/5$, i.e., ≈86%. But across every admission policy the agent tried (doorkeeper generations and thresholds, neighbor co-admission, frequency variants), the hit rate of a cold 30-second window pinned at 76.8–77.1%: at ∼5.5 Mops/s the window simply does not re-reference enough distinct keys often enough for any of them to do better. At h=77% the miss-bandwidth fixed point is $2 \times 0.718/0.23 \approx 6.2$ Mops/s; the engine's 5.51 is about 88% of that bound, the remainder going to write-back I/O contention and CPU. This is an empirical bound for *this class of design* (one 4 KiB read per miss, admission-based caching, 8 GiB of honest memory), supported by a policy sweep; it is not an impossibility proof over all designs. A scheme that co-locates hot records on shared pages could in principle beat it, though our attempt at exactly that scored worse (Section 5.5). Breaking ≈86% within this class would require more effective cache than 8 GiB. The one cheap way to get it, letting the OS page cache inside the 14 GiB cgroup absorb the spill file, was rejected as violating the budget's intent; the engine opens its log O_DIRECT precisely so its disk traffic cannot ride the page cache.

5.4 Correctness, testing, and what the tests caught

The hardening task (Section 6) produced an end-to-end test suite (a 935-line test program plus a 44-line matrix runner) that drives the real engine through real files with scaled workloads—no mocks—covering concurrent read/upsert/delete/checkpoint interleavings, fingerprint-alias key sets crafted by inverting the index hash, oversized-value paths, the `io_uring`-disabled fallback, and delete-resurrection oracles (four readers hammering keys while a writer flaps upsert/delete, with per-key issue/completion interval brackets making "value from the wrong phase" detectable). The matrix runs six configurations: optimized, buffered-I/O on tmpfs, no-`io_uring`, ASan+UBSan, TSan (a nine-suite subset, since TSan's slowdown makes the full suite impractical), and a coverage build; all pass, with 92.84% of engine lines and 93.42% of branches executed at least once under gcov (the uncovered remainder is dominated by fail-stop I/O-error paths and sub-microsecond race windows, per manual inspection of the gcov report). Known residuals the suite does *not* exercise are listed in Section 7. The delete-resurrection oracle is the suite's strongest result: against two successive fixes that looked plausible under review it reliably drove the failure counter to the test's abort threshold within seconds, and it went green only under the pin-ownership protocol of Section 4. A final audit confirmed the engine performs no trace reads and contains no key-population constants.

5.5 Negative results

Discovery-loop bookkeeping (an append-only ideas ledger the agent maintained to avoid retrying failures) records the ideas that measured worse and were reverted, each scored on the original workload against the then-current base, so the numbers are comparable to their own base, not to each other: strict-probation segmentation (0.78 Mops/s; forward progress stalls when probation fills with pinned entries), 2-way probation (0.95), page-neighbor co-admission on read misses (1.98; admitting a missed page's other residents pollutes probation faster than it prefetches), always admitting load-phase-clean keys (4.17), cleaners that flush *any* dirty entry rather than only cold ones (0.47; hot-key reflush storms saturate write bandwidth), and a heat-clustered relocation scheme that groups hot keys onto shared pages (4.03–4.34; the extra write traffic exceeded the locality benefit at this value size). The last entry matters beyond its number: the hardening task's initial 3.6 Mops/s "regres-

sion" turned out to be correctness fixes layered onto this already-rejected relocation branch rather than onto the fastest revision, and only bisection against archived scored revisions exposed it. That episode is the strongest argument we have for keeping an immutable record of what was tried, on which base, at what score.

6 How HydraKV Was Built: a Task Sequence in KISS Sorcar

HydraKV's development is inseparable from its methodology, so we document it as the single case study it is; the process observations below are from this one project, not controlled comparisons. All work ran in KISS Sorcar [15], whose relevant properties here are: long-horizon task execution with automatic continuation across context windows, unattended operation against a remote benchmark machine over SSH, mixing models from different vendors inside one task via prompts, and persistent per-repository progress notes that later sessions read. The human contribution was three task-opening prompts and two shorter mid-task steering messages; the agent wrote every line of engine and test code. We reproduce the three prompts below word for word (typos, model directives, and the author's own server address included) because they *are* the specification the agent worked from; nothing was cleaned up for publication, though TeX re-wraps lines and collapses runs of spaces. (Three pieces of notation recur: `claude-fable-5` and `gpt-5.6-sol` are the identifiers under which the framework ran its development and review models; "e2e" abbreviates end-to-end; `./KV_TASK.md` is a local copy of the benchmark's task specification.) Of the two steering messages, one is quoted in place below; the other survives only as a contemporaneous summary, which we flag when we reach it. Tasks 2 and 3 explicitly split the roles (`claude-fable-5` for development, `gpt-5.6-sol` in a read-only debugging role), and independent reviews ran at the major milestones described below. Total model-API spend across all sessions, summed from the framework's per-session budget accounting, was under $200.

Task 1: baseline and first engine. The first task never says "write a key-value store"; it asks the agent to set up the benchmark repository on the remote machine and run the setup document's steps, which in turn require supplying an engine and scoring it (the setup's pass bar was a validated run with a median above the 0.93 baseline; its stated target, about 2× that):

````
I have a server where I can login using: ssh ksen@34.70.16.69 .
My working directory on the server is /mnt/ssd/ksen/kv50-benchmark .
After logging in can you clone https://github.com/shubham3-ucb/baselines-kiss-sorcar.git using the
credentials of ksenxx user on github (the local machine has the credentials). Then do the following:
```
cd baselines-kiss-sorcar/task
export KV_SPILL=/mnt/ssd/kv_spill
```
Then run the steps 3 to 6 at https://github.com/shubham3-ucb/baselines-kiss-sorcar/blob/main/SETUP.md .
````

The agent produced a first-principles engine in one session: a flat 4-byte-per-key position array (exploiting the published trace's dense key space), fixed 128-byte slots written sequentially at load, in-place 4 KiB read-modify-writes for write-back, and a version-checked write-back cache. It validated (250,000,000/250,000,000 keys) and scored 1.85 Mops/s, just under 2× FASTER. The dense-key assumption also left it heavily overfitted to the published trace, as the next task showed.

Task 2: adversarial AI discovery. The second task installed the loop, in full:

```
Can you perform adversarial AI Discovery for the task at ./KV_TASK.md so that the goals in the task are
met? Look at the previous task on how to validate the engine on the server. You MUST not stop until the
goals are met. Generate adversarial workload variants to make sure that the engine works fast on the
variant workloads. Do the following iteratively while maintaining a variable iteration_count variable
which starts at 1 and increments by 1 on each iteration:
Generate a variant workload that are realistic like the original workload, but breaks the performance
gain of the engine. Do extensive internet search to understand how to make the variant workload
realistic to real-world workloads and robust to reward hacking or cheating. Then run the engine on the
variant workload. If the goals are not met, use AI discovery to improve the engine on all workloads
until you achieve the goals without cheating using the workloads. Then generate a new workload repeat
the process until the engine can achieve the goals on the new test workload on which AI discovery was
not performed.

Search the internet extensively. Use 'claude-fable-5 model' for all tasks, including software
development. Use 'gpt-5.6-sol' (not codex) for a thorough read-only review and debugging of the other
```

```
model's work. Thoroughly check whether the other model has missed any code or wiring or introduced any
bugs. Use at most 20% of task budget in gpt-5.6-sol for reviewing and debugging. Use the model names
literally without hallucinating new model names.
```

The first of the two steering messages arrived mid-loop, raising the goal from $2\times$ FASTER to a flat 10 Mops/s and adding a strict no-hardcoding requirement (no constants derived from the trace's dense key space or key count); its exact wording was not preserved in the session log, so we summarize its content here. The agent grounded its variant axes in the workload-characterization literature it retrieved (key locality at Facebook [4], skew and temporal dynamics at Twitter [5], YCSB's generator design [2]), wrote the variant generator, and executed the loop of Section 5.2: V1 broke the dense index (OOM); the rebuilt fingerprint-index engine passed V1 at 1.91 Mops/s; the goal raise triggered the profiling-driven redesign (log-append write-back, sharded I/O queues, doorkeeper, then `io_uring` rings and segmented insertion) that reached 5.50; V2/V3 and the holdout closed the loop. Two disciplines were enforced by the prompt and are visible in the artifact: the same binary ran every workload, and each accepted idea appears in the ledger with its measured score, each rejected one with its failure number (Section 5.5). The 10 Mops/s goal ended as the ceiling argument of Section 5.3.

Task 3: find every bug, keep the speed. The third task, again in full:

```
can you find all redundancies, inconsistencies, obvious bugs, corner case bugs, race conditions,
deadlocks in your final implementation of the engine? Validate them by writing e2e tests with workloads.
Then remove them and make sure that all tests pass. Write e2e tests with workloads to achieve 100%
coverage of the egine code. Check the correctness of the engine by writing adversarial e2e tests with
workloads. Honestly review the code to check if you have cheated based on the workload knowledge. If so,
remove them. You MUST NOT reduce the performance below 5.48 Mops while fixing bugs. Search the internet
extensively. Use claude-fable-5 model for all tasks, including software development. Use gpt-5.6-sol
(not codex) for a thorough review and debugging of the other model's work. Thoroughly check whether the
other model has missed any code or wiring or introduced any bugs.
```

The prompt's hard floor of 5.48 Mops/s is the v4c score on the original workload. Two of its literal demands were not met: coverage stopped at 92.84% of lines and 93.42% of branches (Section 5.4), and "all" bugs is not something any test suite certifies (Section 7 lists the residual risks left open). This task consumed three sessions and put the most weight on the split between developer and reviewer models. The `gpt-5.6-sol` reviews produced dozens of line-referenced findings, of which the serious ones were: the fingerprint-alias hazard of Section 4 (an effective-signature collision is $\approx$20% likely per run at this key count and, in the pre-fix engine, could abort the process or bury a live key); the observation that slot position is not a valid recency order across pre-reserved extents, which invalidated a chain-walk guard; a delete/in-flight-admission race; and `io_uring` partial-submission accounting. The new delete-resurrection test then falsified two successive epoch-based fixes for the delete race before the pin-ownership protocol survived it. When the assembled fixes scored 3.6 Mops/s, the second steering message supplied the decisive hint, in its entirety: *"Did you start with the most performant variant of the engine?"* The agent bisected archived engine revisions, discovered the answer was no (the fixes had been layered onto the rejected relocation branch), rebased all of them onto the fastest revision, and re-ran the full matrix and the scored gate: 5.51 / 5.51 / 5.53, slightly above the floor; the recovery came from two prefetch hints found during the re-port.

Observations. Three observations from this one case study stood out. (1) *Adversarial self-evaluation was cheap and decisive.* Generating a distribution shift and re-scoring cost minutes and immediately exposed the dense-index overfit; the held-out variant then bounded how much the repairs themselves had overfit the loop. (2) *Cross-model review surfaced different bugs than same-model review.* The reviewer model, prompted read-only with no authorship stake, repeatedly flagged concurrency hazards the developer model had rationalized; the resurrection race was ultimately found by a test the reviewer's findings inspired, not by either model's reading of the code. (3) *Agents need immutable experimental provenance.* Both failure modes hit during development (retrying known-bad ideas, and hardening the wrong engine version) were provenance failures, repaired by artifacts (the ideas ledger, archived scored revisions) that cost almost nothing to maintain.

7 Limitations

HydraKV is a benchmark-shaped engine, and we list where it falls short of the production systems it is compared against, together with the residual risks left open at the end of hardening.

Missing subsystems. The append-only log has no garbage collection or compaction; sustained update, insert, or delete traffic grows the spill file until an offline rewrite, and the 32-bit position space is exhausted at roughly 512 GiB of log. Crash recovery is not implemented: the spill file is truncated at initialization, and there is no metadata snapshot from which the index and cache could be rebuilt; `Checkpoint` is a flush primitive with a quiescent-sessions precondition (Section 4), not a recovery mechanism. FASTER and RocksDB provide both as core features. HydraKV is better described as an engine for a store than as a store.

Known residual risks. The multi-root freshness hazard for same-signature keys whose first inserts race across sessions (Section 4); the 32-bit staging-token wrap horizon; cross-session partially-filled-chunk interactions (a `Checkpoint` or stop cannot drain another live session's partial chunk); RMW-vs-Upsert atomicity under one specific interleaving; return-value semantics of overlapping `Deletes`; unbounded RAM growth of the oversized-value overflow map, which forfeits the memory-budget property for workloads with values above 101 bytes; and formal (not observed) object-lifetime concerns for atomics in `mmap`ed memory. None is reachable by the scored workload; several are reachable through the public interface by workloads the harness does not generate.

Evaluation scope. The adversarial matrix in Table 1 was scored with the v4c engine; only the original workload was re-scored after hardening (at 5.51), so the variant numbers carry the pre-hardening engine's identity; the hardened engine passes all functional tests on variant-style workloads, but we did not re-run the scored matrix. FASTER was measured only on the original workload; no per-variant comparison exists. We report medians with per-run values where recorded, but no dispersion statistics beyond that, and no CPU/I/O utilization breakdowns for the headline runs; the negative-result scores of Section 5.5 are single measurements on evolving bases. Absolute Mops/s are specific to one machine and one 30-second cold-start window; the multiplier over FASTER is a same-box, same-harness, same-workload comparison and we make no claim about other hardware, other value sizes, longer windows, or production traces. Finally, the variants are synthetic stressors from one generator family; surviving them is evidence against the specific overfits they probe, not evidence of production readiness.

8 Related Work

Key-value stores. FASTER [1] is the direct baseline and the closest design: hash index over a log with a memory-resident mutable region. HydraKV differs in making the RAM tier an admission-controlled cache of individual values rather than a log prefix, and in verifying every index hit against the disk-resident key. KVell [3] argued for unsorted, shared-nothing NVMe designs; HydraKV follows its spirit (no sorting, batched I/O) while keeping a shared cache because the skewed workload rewards it. RocksDB [11] and the LSM lineage [10] target a broader operation mix (scans, transactions, compaction-based space reclamation) that this workload does not exercise. Redis [12] serves the in-memory regime and does not natively address a larger-than-memory budget.

Caching. The admission-and-segmentation toolkit HydraKV borrows is from the caching literature: segmented LRU [8], CLOCK-style recency [9], the TinyLFU/W-TinyLFU admission line with its doorkeeper filter [7], and the engineering synthesis in CacheLib [6]. The workload studies that motivated the adversarial variant axes are the Facebook RocksDB characterization [4] (key-space locality, and YCSB's lack of it) and the Twitter cache analysis [5] (write-heavy skew, temporal dynamics). YCSB itself [2] defines the workload family and Zipfian setting used throughout.

AI-driven code discovery. Evolutionary and agentic code-optimization systems score candidate programs against evaluators—AlphaEvolve [16] against automated metrics, GEPA [17] against task scores with explicit train/validation/test splits to control overfitting to the evaluation set. The adversarial workload loop used here is a domain-specific instance of the same concern: it makes the evaluator a moving, realism-constrained target with a held-out final measurement, closer in spirit to adversarial evaluation practice in ML robustness than to static benchmarking. The agent frame-

work, its layered architecture, and its multi-model task execution are described in the KISS Sorcar paper [15]; this paper is a case study of that framework doing sustained systems work.

9 Conclusion

A general-purpose AI agent, steered by three natural-language tasks, produced a $\sim$2,050-line key-value store engine that outperforms FASTER by 5.9$\times$ on a demanding larger-than-memory YCSB-A configuration, together with evidence (adversarial variants, a held-out workload, sanitizer-clean end-to-end tests, and a disclosed list of residual risks) about how far that number generalizes and where it stops. What made the exercise trustworthy was ordinary experimental hygiene, enforced on the agent: distribution shifts with a post-hoc holdout, a hard memory budget with the page-cache loophole explicitly closed, cross-model review with no authorship stake, end-to-end tests under sanitizers, a performance floor on every fix, and an immutable ledger of what was tried on which engine at what score. Where a goal exceeded the measured ceiling of the design class, the agent said so, with arithmetic. We suspect this recipe transfers to most performance engineering an agent might be asked to do; establishing that beyond a single case study is future work.

Acknowledgments

This research is supported in part by gifts from Accenture, Amazon, AMD, Anyscale, Broadcom, Google, IBM, Intel, Intesa Sanpaolo, Lambda, Lightspeed, Mibura, NVIDIA, Samsung SDS, SAP, by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research through the X-STACK: Programming Environments for Scientific Computing program (DESC0021982,) and the Defense Advanced Research Projects Agency (DARPA) under Agreement No. HR00112590134.

References

[1] B. Chandramouli, G. Prasaad, D. Kossmann, J. Levandoski, J. Hunter, and M. Barnett. FASTER: A Concurrent Key-Value Store with In-Place Updates. In *ACM SIGMOD International Conference on Management of Data (SIGMOD)*, 2018.

[2] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears. Benchmarking Cloud Serving Systems with YCSB. In *ACM Symposium on Cloud Computing (SoCC)*, pages 143–154, 2010. DOI 10.1145/1807128.1807152.

[3] B. Lepers, O. Balmau, K. Gupta, and W. Zwaenepoel. KVell: the Design and Implementation of a Fast Persistent Key-Value Store. In *ACM Symposium on Operating Systems Principles (SOSP)*, pages 447–461, 2019. DOI 10.1145/3341301.3359628.

[4] Z. Cao, S. Dong, S. Vemuri, and D. H. C. Du. Characterizing, Modeling, and Benchmarking RocksDB Key-Value Workloads at Facebook. In *USENIX Conference on File and Storage Technologies (FAST)*, 2020.

[5] J. Yang, Y. Yue, and K. V. Rashmi. A Large Scale Analysis of Hundreds of In-Memory Cache Clusters at Twitter. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020.

[6] B. Berg, D. S. Berger, S. McAllister, I. Grosof, S. Gunasekar, J. Lu, M. Uhlar, J. Carrig, N. Beckmann, M. Harchol-Balter, and G. R. Ganger. The CacheLib Caching Engine: Design and Experiences at Scale. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020.

[7] G. Einziger, R. Friedman, and B. Manes. TinyLFU: A Highly Efficient Cache Admission Policy. *ACM Transactions on Storage*, 13(4), 2017. arXiv:1512.00727.

[8] R. Karedla, J. S. Love, and B. G. Wherry. Caching Strategies to Improve Disk System Performance. *IEEE Computer*, 27(3):38–46, 1994.

[9] F. J. Corbató. A Paging Experiment with the Multics System. In *In Honor of Philip M. Morse* (H. Feshbach and K. U. Ingard, eds.), MIT Press, 1969.

[10] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil. The Log-Structured Merge-Tree (LSM-Tree). *Acta Informatica*, 33:351–385, 1996. DOI 10.1007/s002360050048.

[11] Meta Platforms, Inc. RocksDB: A Persistent Key-Value Store for Fast Storage Environments. `https://rocksdb.org`, accessed 2026.

[12] Redis Ltd. Redis: An In-Memory Data Store. `https://redis.io`, accessed 2026.

[13] J. Axboe. Efficient IO with io_uring, 2019; and io_uring(7), Linux Programmer's Manual, `https://man7.org/linux/man-pages/man7/io_uring.7.html`, accessed 2026.

[14] G. L. Steele Jr., D. Lea, and C. H. Flood. Fast Splittable Pseudorandom Number Generators. In *ACM Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 453–472, 2014. DOI 10.1145/2660193.2660195.

[15] K. Sen. KISS Sorcar: A Stupidly-Simple General-Purpose and Software Engineering AI Assistant. `https://github.com/ksenxx/kiss_ai` (paper at `papers/kisssorcar/`), 2026.

[16] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, and M. Balog. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. arXiv:2506.13131, 2025.

[17] L. A. Agrawal, S. Tan, D. Soylu, N. Ziems, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, C. Potts, K. Sen, A. G. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2507.19457.